# Tarea: El Perceptrón

Ciencia de Datos — Universidad Católica del Norte

**Objetivos de esta tarea**

- Implementar en `numpy` la regla de actualización, el bucle de entrenamiento por épocas y su criterio de parada.
- Contar cuántas actualizaciones toma converger y contrastar ese número con la cota teórica $(R/\gamma)^2$, viendo cómo se comporta cuando el margen se reduce.
- Constatar experimentalmente que el algoritmo no converge sobre XOR, y conectar ese fracaso con la necesidad de modelos no lineales: capas ocultas.
- Dibujar la frontera de decisión que produce un vector de pesos, tanto en el plano original como en el espacio de la capa oculta.

## Contexto

En clases vimos el perceptrón como una **unidad de umbral**: dispara cuando la suma ponderada de sus entradas supera un umbral $T$. También vimos el truco que simplifica toda la notación: si agregamos a cada ejemplo una coordenada fija igual a $1$, con peso $-T$, entonces

$$\sum_{i=1}^{N} w_i x_i \geq T \quad \Longleftrightarrow \quad \sum_{i=1}^{N+1} w_i x_i \geq 0 \quad \Longleftrightarrow \quad \mathbf{w} \cdot \mathbf{x} \geq 0.$$

En ese **espacio aumentado** la frontera es un hiperplano que pasa por el origen, y el vector $\mathbf{w}$ es perpendicular a ella y apunta hacia la clase positiva. Toda la tarea trabaja así: cuando el problema necesita umbral, la última columna de los datos es una columna de unos y el umbral es una coordenada más de $\mathbf{w}$. Cuando no lo necesita (porque la frontera ya pasa por el origen), esa columna simplemente no está.

Con etiquetas $y \in \{-1, +1\}$, el **puntaje** de un ejemplo es $\mathbf{w} \cdot \mathbf{x}$ y su **margen** es $(\mathbf{w} \cdot \mathbf{x})\, y$. Igual que en la clasificación lineal que ya vimos: margen positivo significa que la predicción acierta, y su magnitud dice cuán lejos de la frontera quedó el punto.

### La regla del perceptrón

**Algoritmo del perceptrón (Rosenblatt).** Inicializar $\mathbf{w} = \mathbf{0}$. Repetir sobre los ejemplos $(\mathbf{x}, y)$:

$$\text{si } (\mathbf{w} \cdot \mathbf{x})\, y \leq 0 \text{ (mal clasificado):} \qquad \mathbf{w} \leftarrow \mathbf{w} + y\,\mathbf{x}.$$

Detenerse cuando una pasada completa no produzca ninguna actualización.

Fíjate en tres detalles que van a importar durante toda la tarea:

- **Solo se actualiza con los ejemplos mal clasificados.** Los que ya están bien no mueven nada. Esto no es descenso por gradiente sobre una pérdida suave: es una regla de corrección de errores.

- **El margen exactamente $0$ cuenta como error.** Un punto sobre la frontera no está bien clasificado. Esta convención es la que permite que el algoritmo arranque: partiendo de $\mathbf{w} = \mathbf{0}$ todos los márgenes valen $0$ y por lo tanto el primer ejemplo siempre actualiza.
- **La actualización gira $\mathbf{w}$ hacia el ejemplo que falló.** Si el ejemplo mal clasificado es positivo, se le suma $\mathbf{x}$ a $\mathbf{w}$; si es negativo, se le resta.

## La garantía de convergencia

El teorema de convergencia del perceptrón dice: si las clases son **linealmente separables**, el algoritmo (partiendo de $\mathbf{w} = \mathbf{0}$) hace a lo más

$$\left(\frac{R}{\gamma}\right)^2 \qquad \text{actualizaciones,}$$

donde $R = \max_i \|\mathbf{x}_i\|$ es la norma del ejemplo más largo y $\gamma$ es el **margen geométrico** del mejor separador: la distancia del punto más cercano a esa frontera. Formalmente, si $\mathbf{w}_{\text{ref}}$ separa las clases,

$$\gamma = \min_i \frac{(\mathbf{w}_{\text{ref}} \cdot \mathbf{x}_i)\, y_i}{\|\mathbf{w}_{\text{ref}}\|}.$$

La intuición es que deshacer un error de tamaño $R$ toma muchos pasos de tamaño $\gamma$. Nota que la cota *no* depende de la cantidad de ejemplos ni de la dimensión: solo de la geometría del problema. Nota también lo que **no** dice: nada sobre qué pasa cuando las clases no son separables.

## Archivos

- `submission.py`: el único archivo que debes modificar y el único que se entrega. Trae dos cosas: la **sección de respuestas** del comienzo, donde anotas los números de los experimentos, y las cinco funciones de la tarea, cada una con su contrato en el docstring.
- `util.py`: conjuntos de datos y utilidades. Ya está completo y no se toca. Fija el backend `Agg` de matplotlib, así que los gráficos se guardan en archivos y nunca se abre una ventana.
- `experimento.py`: el cierre de la tarea. Corre los tres experimentos, deja en la terminal los números de los que sale la sección de respuestas y guarda los gráficos en disco. Tampoco se modifica.
- `grader.py`: corrector automático. Corre sin conexión a internet con `python grader.py`.
- `requirements.txt`: `numpy` y `matplotlib`. Nada más.

Todos los datos se generan localmente con semillas fijas: la tarea no descarga nada y dos corridas seguidas dan exactamente lo mismo.

## La sección de respuestas

Arriba de todo en `submission.py` hay una sección de constantes con los números que salen de correr tus propias funciones. No se calculan a mano ni se copian de este enunciado: se leen de lo que imprime `python experimento.py` cuando la parte correspondiente ya está lista. Cada constante lleva encima el experimento y la columna de la que sale, y cada problema dice cuáles le tocan.

Esa sección se corrige junto con el código, buscando cada constante por su nombre, así que no cambies los nombres ni escribas texto adentro de los valores. Las reglas de comparación son las mismas en toda la tarea:

- los conteos (épocas y actualizaciones) son enteros exactos;
- la cota se compara con **un decimal** y el radio $R$ con **tres**, la misma precisión con la que los imprime `experimento.py`;
- las claves de cada diccionario ya vienen puestas y tienen que quedar tal cual, ni una más ni una menos.

**Qué entregar**

- **Entrega**: un solo archivo, `submission.py`, con las cinco funciones completadas y la sección de respuestas llena. No se entrega nada más (ni informe, ni gráficos, ni un `.zip`), y no cambies las firmas ni los nombres de las constantes, porque el corrector los llama tal como están escritos.
- Antes de entregar, ejecuta `python grader.py` en la carpeta de la tarea: debe reportar el puntaje completo de las pruebas visibles, que suman 60 de los 100 puntos. Llegar a ese total es condición *necesaria y no suficiente*, porque la nota la pone el corrector del curso, que corre esas mismas pruebas, otras sobre las mismas cinco funciones y la revisión de la sección de respuestas.
- Puntaje total: 100 puntos, repartidos en los tres problemas que vienen a continuación. Los 100 se corrigen solos, sin revisión a mano.
- Los gráficos que deja `experimento.py` son trabajo tuyo para entender la tarea y no se entregan: lo que se corrige es la función que los produce, y el corrector la ejecuta por su cuenta.
- No incluyas la carpeta `__pycache__`.

---

## Problema 1: La regla del perceptrón en numpy [47 puntos]

Todo este problema va en `submission.py`. Las convenciones están en el docstring del archivo y son las mismas del enunciado: `X` de forma $(n, d)$ con una fila por ejemplo, `y` de forma $(n,)$ con valores en $\{-1, +1\}$, `w` de forma $(d,)$, y un ejemplo mal clasificado cuando su margen es **menor o igual que cero**.

### (a) Márgenes [8 puntos]

Implementa `margenes(X, y, w)`, que devuelve el arreglo de forma $(n,)$ con el margen $(\mathbf{w} \cdot \mathbf{x}_i)\, y_i$ de cada ejemplo, en el orden de las filas de `X`. No debe modificar los arreglos que recibe.

### (b) La regla de actualización [16 puntos]

Implementa `paso_perceptron(x, y, w)`, que aplica la regla a un único ejemplo y devuelve la tupla `(w_nuevo, actualizo)`. El segundo elemento es un `bool` que dice si hubo actualización. La función debe devolver un arreglo **nuevo**: no puede modificar `w` en el lugar.

### (c) El bucle de entrenamiento [23 puntos]

Implementa `entrenar_perceptron(X, y, max_epocas=100, w0=None)`. Una **época** es una pasada completa sobre las filas de `X`, en el orden en que vienen, aplicando la regla a cada ejemplo; las actualizaciones se acumulan dentro de la misma época, o sea que el segundo ejemplo ya ve el `w` corregido por el primero. El criterio de parada es que una época completa termine **sin ninguna actualización**; el tope `max_epocas` es lo que evita un bucle infinito cuando los datos no son separables. Devuelve un diccionario con exactamente las claves `"w"`, `"actualizaciones"`, `"epocas"` y `"convergio"`, descritas en el docstring.

**Pista**

Las tres funciones son cortas: la primera es una línea, la segunda son tres y la tercera es un doble `for` con un contador. Si te tienta escribir `w += y * x`, no lo hagas: eso modifica el arreglo del llamador y el corrector lo detecta. Y ojo con contar las épocas: la última, la que *no* actualiza, también cuenta.

## Problema 2: Cuántas actualizaciones, y la cota teórica [28 puntos]

### (a) Implementar la cota [12 puntos]

Implementa `cota_actualizaciones(X, y, w_ref)`, que devuelve $(R/\gamma)^2$ con $R = \max_i \|\mathbf{x}_i\|$ y $\gamma = \min_i (\mathbf{w}_{\text{ref}} \cdot \mathbf{x}_i)\, y_i / \|\mathbf{w}_{\text{ref}}\|$. Debe lanzar `ValueError` si `w_ref` es el vector cero o si no separa las clases (es decir, si $\gamma$ no resulta estrictamente positivo): en ese caso el teorema no dice nada y devolver un número sería mentir.

**Pista**

La división por $\|\mathbf{w}_{\text{ref}}\|$ no es decorativa: sin ella, el resultado cambiaría al multiplicar `w_ref` por 10, y la cota es una propiedad geométrica del conjunto, no de la escala del vector que uses para describirla. El corrector prueba exactamente eso. Para los cinco ejemplos de `datos_tarea()` con $\mathbf{w}_{\text{ref}} = (1, 0)$ debes obtener $5{,}0$.

### (b) La tabla del experimento [16 puntos]

Con las cinco funciones completas, corre `python experimento.py`. Su primer experimento entrena sobre conjuntos separables de 120 puntos con márgenes cada vez más pequeños e imprime una tabla de cuatro filas, una por margen, con el radio $R$, las actualizaciones reales y la cota.

Anota esa tabla en la **sección de respuestas** de `submission.py`. Son tres constantes, una por columna, con el margen de la fila como clave y las cuatro claves ya puestas:

- `TABLA_ACTUALIZACIONES`: la columna `actualiz.`, en enteros.
- `TABLA_COTA`: la columna `cota`, con un decimal.
- `TABLA_RADIO`: la columna `R`, con tres decimales.

Los números salen de tu propio código: `TABLA_ACTUALIZACIONES` la produce tu `entrenar_perceptron` y `TABLA_COTA`, tu `cota_actualizaciones`. Los conjuntos se generan con semilla fija, de modo que la tabla es la misma en cualquier computador.

## Problema 3: XOR y la necesidad de capas ocultas [25 puntos]

### (a) Graficar la frontera de decisión [10 puntos]

Implementa `graficar_frontera(X, y, w, ruta, titulo=...)`, que guarda en `ruta` un archivo PNG con los puntos de cada clase (marcadores distintos), la frontera $\mathbf{w} \cdot \mathbf{x} = 0$, los ejes rotulados, el título recibido y una leyenda. La función no devuelve nada: su efecto es dejar el archivo escrito. Debe funcionar tanto con `X` de dos columnas (frontera por el origen) como de tres (la tercera es la columna de unos y `w[2]` es el umbral). Cierra la figura con `plt.close(fig)` al terminar.

**Pista**

Importa `plt` desde `util` (`from util import plt`): ahí ya está fijado el backend `Agg`, que es el que guarda en archivo sin abrir ninguna ventana. Para la frontera, en vez de despejar $x_2$ (que se cae cuando $w_2 = 0$), evalúa el puntaje sobre una malla con `np.meshgrid` y pinta los dos semiplanos con `ax.contourf`; una llamada a `ax.contour(..., levels=[0.0])` agrega la recta. Guarda con `fig.savefig(ruta)`.

### (b) Los números del experimento de XOR [15 puntos]

El tercer experimento de `python experimento.py` entrena sobre los cuatro ejemplos de XOR en el espacio aumentado con tres topes de épocas distintos, imprime además el detalle paso a paso de los pesos dentro de una época (vale la pena seguirlo, aunque no se pida anotarlo) y después repite el entrenamiento sobre la representación $(h_1, h_2, 1)$ que produce la capa oculta fija de dos unidades de umbral.

Anota en la **sección de respuestas** de `submission.py`:

- `XOR_ACTUALIZACIONES`: las actualizaciones que aplica sobre XOR con cada uno de los tres topes, con el `max_epocas` como clave.
- `XOR_OCULTA_EPOCAS` y `XOR_OCULTA_ACTUALIZACIONES`: las épocas y las actualizaciones del entrenamiento sobre la representación oculta, el que sí converge.
- `XOR_OCULTA_W`: las tres coordenadas del `w final` de ese mismo entrenamiento, en el orden en que las imprime el programa.